

Seksjon 1

Om eksamen

DAT108 Programmering og Webapplikasjoner, vår 2025

Tid: 4 timer

Du kan svare på engelsk eller norsk. Skriver du på norsk kan du bruke engelske faguttrykk.

Jeg har lest og forstått instruksjonene

☐ Ja

☐ Nei

Oppgave 1 (10%) - Strømmer/Streams

Kort om retting:

- Teller ca. 10% (24 minutter).
- Fire deloppgaver: A, B, C og D.
- Hver deloppgave teller ca. 2,5% (6 minutter)
- Oppgaven rettes automatisk.
- Det er bare en **helt** rett løsning på hvert problem.
- Du blir gitt **delvis** uttelling for delvis korrekt svar:
 - Alle felt som er korrekte gir poeng.
 - Feil svar gir **IKKE** trekk.

Oppgave A:

Les hver beskrivelse nøye og velg det svaret som best passer til oppgaven.

i) Hvilken metode er det mest naturlig å bruke for å gjøre om en `Stream<T>` til en enkelt **boolsk** verdi som indikerer om **alle** elementer tilfredsstiller en betingelse?

a `allMatch`

b `anyMatch`

c `noneMatch`

d `filter`

e `reduce`

ii) Hva gjør kallet `stream().limit(4)` ?

a Begrenser strømmen slik at kun de fire første elementene blir med videre i stream-operasjonene.

b Hopper over de fire første elementene og behandler resten.

c Sorterer de fire første elementene.

d Dupliserer de fire første elementene.

e Fjerner alle instanser av verdien 4 fra strømmen.

iii) Hvordan kan du konvertere en `Stream<Integer>` til en `IntStream` for primitive operasjoner?

- a `mapToInt(i -> i)`
- b `map(i -> i)`
- c `map(Integer::intValue)`
- d `flatMapToInt(...)`
- e `map(x -> (int) x)`

Hvilken beskrivelse passer best for metoden `filter` i strømmer?

- a Terminerer strømmen og logger alle elementene.
- b Legger til rette for sideeffekter gjennom et mellomledd, uten å påvirke datastrømmen.
- c Beholder de elementene i strømmen som oppfyller en betingelse.
- d Kombinerer alle elementene til én sluttverdi ved hjelp av en akkumuleringsfunksjon.
- e Bestemmer det største elementet i strømmen ved hjelp av en sammenligningsfunksjon.

v) Hva gjør metoden `findFirst()` når den brukes på en `Stream<T>` ?

- a Returnerer en `List<T>` som inneholder det første elementet i strømmen.
- b Returnerer en `T`, eller kaster unntak hvis strømmen er tom.
- c Returnerer en `Optional<T>` som inneholder det første elementet, eller er tom hvis strømmen er tom.
- d Filtrerer ut det første elementet fra strømmen og sender resten videre.
- e Returnerer en strøm som inneholder kun det første elementet.

Oppgave B:

Du skal behandle en liste med målinger fra en sensor, gitt som `List<Double>` målinger. Databehandlingen skal følge disse stegene:

1. Først fjernes støy ved å kun beholde målinger med en verdi på 2.5 eller høyere.
2. Hver gyldige måling må deretter kalibreres ved å multiplisere den med en faktor på 1.6.
3. For lagring må hver kalibrerte verdi avrundes til to desimaler.
4. Bare de fem første gyldige og kalibrerte målingene skal brukes i den endelige beregningen.
5. Til slutt skal gjennomsnittet av disse fem verdiene beregnes. Hvis ingen målinger er gyldige, skal resultatet være -1.0;

```
Double resultat = maalinger.stream()  
    .  
    .  
    .  
    .  
    .  
    . ;
```

Oppgave C:

Du har en liste `List<Ordre>`, der hver ordre inneholder en vare og ønsket antall.

Du skal behandle ordrene ved å først filtrere bort de som ikke kan leveres fra lageret (lageret må ha nok enheter). Deretter sorteres de gjenværende ordrene etter antall, med flest først. Hver ordre gjøres så om til en `Pakke`. Pakkene som veier mer enn 30 kg filtreres bort, og de som gjenstår sorteres etter vekt, lettest først. Til slutt **registreres** pakkene for transport.

Du har følgende hjelpeobjekter:

```
Predicate<Ordre> kanLeveres;           // true hvis lageret har nok av varen  
Comparator<Ordre> etterAntall;        // sorterer ordrer etter antall, synkende  
Function<Ordre, Pakke> tilPakke;      // lager en Pakke basert på en Ordre  
Predicate<Pakke> erSendbar;           // true hvis pakken veier maks 30 kg  
Comparator<Pakke> etterVekt;          // sorterer pakker etter vekt, stigende  
Consumer<Pakke> registrer;            // registrerer pakken for transport
```

A

```
ordrer.stream()  
    .filter(kanLeveres)  
    .sorted(etterAntall)  
    .map(tilPakke)  
    .filter(erSendbar)  
    .sorted(etterVekt)  
    .forEach(registrer);
```

C

```
ordrer.stream()  
    .filter(kanLeveres)  
    .sorted(etterAntall)  
    .map(tilPakke)  
    .filter(erSendbar)  
    .sorted(etterVekt)  
    .toList();
```

B

```
ordrer.stream()  
    .sorted(etterAntall)  
    .filter(kanLeveres)  
    .map(tilPakke)  
    .filter(erSendbar)  
    .sorted(etterAntall)  
    .forEach(registrer);
```

D

```
ordrer.stream()  
    .filter(kanLeveres)  
    .map(tilPakke)  
    .filter(erSendbar)  
    .sorted(etterVekt)  
    .toList();
```

Oppgave D: Hvilket tall blir skrevet ut når koden under kjøres?

```
import java.util.Arrays;
import java.util.List;

public class StreamExample {
    public static void main(String[] args) {
        List<Integer> tall = Arrays.asList(5, 3, 6, 10, 9, 7, 7, 6, 7, 1);

        double resultat = tall.stream()
            .limit(7)
            .filter(n -> n % 2 == 0)
            .mapToInt(n -> n / 2)
            .sorted()
            .average()
            .orElse(-1);

        System.out.println(resultat);
    }
}
```

Tallet som blir skrevet ut er

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:

// Oppgave B iii).

Dine antagelser...

B *I* U |   á

0 / 10000 Word Limit

Oppgave 2 (10%) - Funksjonelle kontrakter

Kort om retting:

- Teller ca. 10% (24 minutter).
- Fire deloppgaver: A, B, C og D
- Hver deloppgave teller ca. 2.5% (6 minutter)
- Oppgaven rettes automatisk.
- Det er bare en **helt** rett løsning på hvert problem.
- Du blir gitt **delvis** uttelling for delvis korrekt svar:
 - Alle felt som er korrekte gir poeng.
 - Feil svar gir **IKKE** trekk.

Oppgave A: Velg rett type til hver av variablene a1, a2, a3, a4 og a5:

i)

```
a1 = () -> LocalDate.now();
```

A

Supplier<LocalDate>

D

LocalDate

B

Function<Void, LocalDate>

E

Consumer<LocalDate>

C

Runnable

F

() -> LocalDate

ii)

```
a2 = tekst -> tekst.trim().length();
```

A

Function<String, Integer>

D

Predicate<String>

B

Function<int, String>

E

Function<String, String>

C

Consumer<String>

F

String -> Integer

iii)

```
a3 = (liste, verdi) -> liste.contains(verdi);
```

A

BiPredicate<List<String>, String>

D

BiConsumer<List<String>, String>

B

Predicate<List<String>>

E

Function<List<String>, Boolean>

C

BiFunction<List<String>, boolean, String>

F

BiPredicate<Integer, List<String>>

iv)

```
a4 = tall -> tall % 2 == 0 ? "partall" : "oddetall";
```

A

Function<Integer, String>

D

Function<String, int>

B

Predicate<Integer>

E

UnaryOperator<Integer>

C

Function<Integer, Boolean>

F

Consumer<Integer>

v)

```
a5 = liste -> liste.stream().reduce(Integer::sum);
```

A

Function<List<Integer>, Optional<Integer>>

D

Function<List<Integer>, Double>

B

Function<Integer, List<Integer>>

E

UnaryOperator<List<Integer>>

C

BinaryOperator<Integer>

F

UnaryOperator<Optional<List<Integer>>>

Oppgave B: Velg det lambdauttrykket som passer med typen til hver av variabelene b1, b2, b3, b4 og b5.

i)

```
Consumer<String> b1;
```

A

```
System.out::println
```

D

```
s.isEmpty()
```

B

```
() -> ""
```

E

```
(s1, s2) -> s1.concat(s2)
```

C

```
s -> s.toUpperCase()
```

F

```
s -> s.length()
```

ii)

```
Supplier<List<String>> b2;
```

A

```
() -> new ArrayList<>()
```

D

```
() -> "en liste"
```

B

```
s -> List.of(s)
```

E

```
(s1, s2) -> List.of(s1, s2)
```

C

```
list -> list.isEmpty()
```

F

```
list -> System.out.println(list)
```

iii)

```
Comparator<String> b3;
```

A

```
(s1, s2) -> s1.length() - s2.length()
```

D

```
String::equals
```

B

```
(s1, s2) -> s1.length() > s2.length()
```

E

```
(s1, s2) -> s1 + s2
```

C

```
s -> s.length()
```

F

```
() -> 0
```

iv)

```
BinaryOperator<String> b4;
```

A

```
(s1, s2) -> s1.concat(s2)
```

D

```
(s, i) -> s.length() + i
```

B

```
(s1, s2) -> s1.equals(s2)
```

E

```
(s1, s2) -> s1.compareTo(s2)
```

C

```
s -> s + s
```

F

```
(s) -> new String(s)
```

v)

```
BiFunction<String, String, Integer> b5;
```

A

```
(s1, s2) -> 0
```

D

```
(s, i) -> s.length() + i
```

B

```
String::length
```

E

```
Integer::parseInt
```

C

```
(s1, s2) -> s1.equals(s2)
```

F

```
String::concat
```

Oppgave C: Velg rett lambdauttrykk og type til variablene c1, c2 og c3 som passer med beskrivelsen.

For de to siste oppgavene, anta at det finnes en klasse Bok. Et objekt av denne klasse n har følgende informasjon, som er tilgjengelig via "getter"-metoder:

- En tittel (type String), hentes med getTittel()
- En forfatter (type String), hentes med getForfatter()
- Et antall sider (type int), hentes med getAntallSider()
- En boolean-verdi som er true hvis boken er en pocket-utgave, hentes med isPocket()

```
// i)
// Inn: En bok (Bok).
// Ut: 'true' dersom boken er en pocket-utgave med mer enn 400 sider, ellers 'false'.
```

c1 = ;

```
// ii)
// Lag et lambda uttrykk som sammenligner to bøker. Bøker med færre sider skal regnes
som «mindre enn» bøker med flere sider. Dersom to bøker har likt sidetall, skal de sam
menlignes alfabetisk etter tittel.
```

c2 =

;

```
// iii)
// Definer et lambdauttrykk som skriver ut en "etikett" for en bok til konsollen på fo
rmatet: "*Tittel*, av *Forfatter*" (*Tittel* og *Forfatter* erstattes av passende verd
ier).
```

c3 =

;

Oppgave D: Er kodelinjene under gyldig? (I.e. er syntaks korrekt OG er uttrykket gitt rett funksjonell kontrakt)?

		Gyldig	Ugyldig
a	<pre>Predicate<Bok> erKort = bok -> bok.getAntallSider();</pre>	☐	☐
b	<pre>Function<String, Bok> lagBok = tittel -> new Bok(tittel, "Ukjent", 150);</pre>	☐	☐
c	<pre>Comparator<Bok> etterTittel = (b1, b2) -> b1.getTittel().equals(b2.getTittel());</pre>	☐	☐
d	<pre>BiConsumer<Bok, String> oppdaterForfatter = (bok, ny) -> bok.setForfatter(ny);</pre>	☐	☐
e	<pre>UnaryOperator<Bok> rediger = (bok, tittel) -> new Bok(tittel, "N.N.", 200);</pre>	☐	☐
f	<pre>Consumer<Integer> hentSidetall = () -> bok.getAntallSider();</pre>	☐	☐

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:

// Oppgave B iii).

Dine antagelser...

B	<i>I</i>	<u>U</u>		:≡	¹/₃≡	á
0 / 10000 Word Limit						

Oppgave 3 (10%) - JavaScript teorispørsmål

I disse flervalgsoppgavene er ett og kun ett av utsagnene korrekt. En oppgave gir kun poeng hvis det korrekte utsagnet er valgt.

a) (1%)

JavaScript-koden under er brukt i en web-applikasjon:

```
const buttonElement = document.querySelector("button");
buttonElement.addEventListener("click", rullterning);
```

Hva slags størrelse må *rullterning* være for at koden over skal fungere etter hensikten?

- ☐ Et objekt.
- ☐ En forekomst av **Class**.
- ☐ En *function*
- ☐ En forekomst av **Array**.
- ☐ Et **Symbol**
- ☐ En forekomst av **Map**

b) (1%)

I JavaScript har HTML-elementer både egenskapen *innerText* og *textContent*. Hvilket utsagn nedenfor er riktig?

- ☐ Egenskapen *textContent* returnerer alt tekstinnhold i elementet, mens *innerText* returnerer kun synlig innhold i elementet. Ved tilordning oppfører de seg likt.
- ☐ De gjør akkurat det samme, men *textContent* gjelder generelt for XML dokument noder, mens *innerText* er en nyere konstruksjon og kun for HTML elementer.
- ☐ Egenskapen *innerText* brukes for å lese tekstinnhold i elementet, mens *textContent* brukes kun for å legge inn nytt tekstinnhold.
- ☐ Egenskapen *innerText* opererer kun på tekstinnhold som er direkte barn av elementet, mens *textContent* representerer alt tekstinnhold i hele DOM-strukturen under elementet.
- ☐ Egenskapen *innerText* ufarliggjør HTML-koder i teksten, mens *textContent* vil tolke HTML-koder. Derfor bør *innerText* benyttes heller enn *textContent* for å sikre applikasjonen mot XSS angrep.
- ☐ Egenskapen *innerText* må brukes på *HEAD* delen av dokumentet, mens *textContent* på *BODY*.

c) (1%)

Et HTML dokument har følgende *BODY*:

```
<BODY>
  <H1>Velkommen til <SPAN>DAT108</SPAN></H1>
</BODY>
```

Dokumentet inneholder også JavaScript-koden nedenfor:

```
const elementer = document.body.getElementsByTagName("*");
```

Hvor kan JavaScript-koden over plasseres for at koden ikke skal feile?

- ☐ I en tagg *SCRIPT* i *HEAD* delen av dokumentet.
- ☐ I en tagg *JAVASCRIPT* i *HEAD* delen av dokumentet.
- ☐ I en tagg *SCRIPT* i starten av dokumentet sin *BODY*.
- ☐ I en tagg *JAVASCRIPT* i starten av dokumentet sin *BODY*.
- ☐ I en tagg *SCRIPT* i slutten av dokumentet sin *BODY*.
- ☐ I en tagg *JAVASCRIPT* i slutten av dokumentet sin *BODY*.

d) (1%)

Et HTML dokument har følgende *BODY*:

```
<BODY>
  <H1>Velkommen til <SPAN>DAT108</SPAN></H1>
</BODY>
```

Dokumentet inneholder også JavaScript-koden nedenfor:

```
const elementer = document.body.getElementsByTagName("*");
```

JavaScript-koden over er plassert i dokumentet riktig slik at den ikke feiler.

Hvilket av utsagnene under er riktig?

- ☐ Konstanten *elementer* vil ha verdien *null* da tagg «*» ikke er benyttet i dokumentet.
- ☐ Konstanten *elementer* vil ha verdien 2 da det er to HTML elementer i *BODY*.
- ☐ Konstanten *elementer* vil være en statisk **HTMLCollection** av objekter som representerer taggene *H1* og *SPAN*.
- ☐ Konstanten *elementer* vil være en levende **HTMLCollection** av objekter som representerer taggene *H1* og *SPAN*.
- ☐ Konstanten *elementer* vil være en statisk **HTMLCollection** med et objekt som representerer taggen *H1*.
- ☐ Konstanten *elementer* vil være en levende **HTMLCollection** med et objekt om representerer taggen *H1*.

e) (1%)

JavaScript-koden under er brukt i en web-applikasjon:

```
const person = {navn: "Ole", dato: new Date(1998,9,12)};
```

Hvilket av utsagnene under er riktig?

- ☐ Koden over er et eksempel på JSON. JSON er ikke støttet direkte i JavaScript, men det finnes JavaScript bibliotek som muliggjør bruk av JSON.
- ☐ Koden vil feile da navn på egenskap må omslutes av hermetegn.
- ☐ Koden vil feile da forekomst av **Date** ikke er tillatt med JSON.
- ☐ Koden vil fungere, men da **Date** ikke er tillatt med JSON vil *dato* bli tilordnet en tekst-representasjon av **Date** objektet.
- ☐ Koden vil fungere uten problemer og oppretter en forekomst av **Object** der egenskapen *dato* vil være en forekomst av **Date**.
- ☐ Koden vil fungerer uten problemer og oppretter en forekomst av **Class** der egenskapen *dato* vil være en forekomst av **Date**.

f) (1%)

JavaScript-koden under er brukt i en web-applikasjon:

```
let tabell = [ 2, 4, 6, 1 ];  
[,,...tabell] = tabell;
```

Hvilket av utsagnene under er korrekt?

- ☐ Koden vil feile da konstruksjonen «[]» kun tillates på høyre side av likhetstegnet.
- ☐ Koden vil feile da *tabell* har færre enn 5 elementer.
- ☐ Koden vil fjerne de tre første elementene i *tabell*.
- ☐ Koden vil fjerne de to første elementene i *tabell*.
- ☐ Koden vil fjerne de tre siste elementene i *tabell*.
- ☐ Koden vil fjerne de to siste elementene i *tabell*.

g) (1%)

JavaScript-koden under er brukt i en web-applikasjon:

```
const id = 121;  
const navn = "Ole" ;  
const person = { navn, id };
```

Hvilket av utsagnene under er korrekt?

Koden vil opprette en forekomst av **Object** med innhold:

☐

```
{ "Ole", 121 };
```

Koden vil opprette en forekomst av **Object** med innhold:

☐

```
{ "navn": "Ole", "id": 121 };
```

Koden vil opprette en forekomst av **Object** med innhold:

☐

```
{ "Ole": 121 };
```

Koden vil opprette en forekomst av **Map** med innhold;

☐

```
{ "Ole", 121 };
```

Koden vil opprette en forekomst av **Map** med innhold:

☐

```
{ "navn": "Ole", "id": 121 };
```

☐ Koden vil feile da egenskaper må ha et navn.

h) (1%)

JavaScript-koden under er brukt i en web-applikasjon:

```
const tallobjekt = {
  liste: [1, 2],
  sum: function() {
    return this.liste.reduce((acc,curr) => acc+curr);
  }
};

const tallstreng = JSON.stringify(tallobjekt);
```

Slik *reduce* er brukt i koden over vil den returnere summere av elementene i listen.

Hvilket av utsagnene under er korrekt?

- ☐ Koden vil feile ved opprettelsen av konstanten *tallobjekt* da JSON ikke kan inneholde funksjoner.
- ☐ Koden vil feile ved opprettelsen av konstanten *tallstreng* da JSON ikke kan inneholde funksjoner.

Konstanten *tallstreng* vil inneholde tekststrengen:

- ☐ `'{"liste":[1,2], "sum": 3}'`

Konstanten *tallstreng* vil inneholde tekststrengen:

- ☐ `'{"sum": 3}'`

Konstanten *tallstreng* vil inneholde tekststrengen:

- ☐ `'{"liste":[1,2]}'`

Konstanten *tallstreng* vil inneholde tekststrengen:

- ☐ `'{"liste":[1,2], "sum": [1,2].reduce((acc,curr) => acc+curr)}'`

i) (1%)

JavaScript-koden under er brukt i en web-applikasjon:

```
const pElm = document.querySelector("P");
pElm.classList.add("Q");
```

Hva representerer argumentet «Q» i funksjonskallet på siste linje ovenfor?

- ☐ En verdi i *class* attributtet til HTML elementet representert av *pElm*.
- ☐ En class selektor for et «P» element i et CSS dokument.
- ☐ Et HTML-attributt «Q» som skal legges til HTML elementet representert av *pElm*.
- ☐ En JavaScript klasse «Q» der *pElm* legges til som et felt.
- ☐ Et HTML element med tagg «Q» som skal legges til som siste barn av *pElm*.
- ☐ Et HTML element med tagg «Q» som skal legges til som første barn av *pElm*.

j) (1%)

JavaScript-koden under er brukt i en web-applikasjon:

```
if ( tall === 5 ) {  
    console.log("Likhet");  
}
```

Hvilket av utsagnene under er korrekt om operator «===»?

- ☐ Koden vil feile da operator for likhet er «==».
- ☐ Den gjør akkurat det samme som «==», dvs. tester for likhet.
- ☐ Den tester om operandene har samme type uansett verdi.
- ☐ Den tester om operandene har samme verdi uansett type.
- ☐ Den tester om operandene har samme verdi, men ulik type.
- ☐ Den tester om operandene har både samme verdi og samme type.

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:

// Oppgave b).

Dine antagelser...

B <i>I</i> <u>U</u>   á
0 / 10000 Word Limit

Oppgave 4 (15%) - Tråder og trådsikkerhet

Se oppgave på papir. For å svare, velger du «Administrer vedlegg». Så

Nytt vedlegg
Kode
Kodespråk - Java

Da får du åpnet en ny fane der du skriver koden.

For å sikre automatisk lagring, følg denne oppskriften

- 1. Start med å gi vedlegget et navn. For eksempel "Oppgave 4"
- 2. Klikk lagre. Da aktiverer du automatisk lagring.
- 3. Start å skrive kode

Når du har svart, går du tilbake til hovedfanen.

B

I

U

☰

☰
1/2

á

0 / 10000 Word Limit

Oppgave 5 (15%) - JavaScript program

Se oppgave på papir. For å svare, velger du «Administrer vedlegg». Så

Nytt vedlegg

Kode

Kodespråk - JavaScript

Da får du åpnet en ny fane der du skriver koden.

For å sikre automatisk lagring, følg denne oppskriften

1. Start med å gi vedlegget et navn. For eksempel "Oppgave 5"
2. Klikk lagre. Da aktiverer du automatisk lagring.
3. Start å skrive kode

Når du har svart, går du tilbake til hovedfanen.

B	<i>I</i>	<u>U</u>		☰	¹ / ₂ ☰	á
10000 Word Limit						

Kode (som du kan klippe og lime inn i besvarelsen din)

I oppgaven skal dere fylle inn den manglete koden i en JavaScript-klasse **Vennkontroller**. Se oppgavetekst på papir for detaljer.

```
class Vennkontroller {
  #navnefelt;
  #alderfelt;
  // Fyll inn evt. flere felt

  constructor(felt1, felt2) {
    // Kode for felt1, felt for å registrere venn
    this.#navnefelt = felt1.getElementsByTagName("input")[0];
    this.#alderfelt = felt1.getElementsByTagName("input")[1];
    felt1.querySelector("button").addEventListener(
      "click",
      () => { this.#registrervenn() }
    );

    // Legg inn evt. mere kode her
  }

  #registrervenn() {
    /* Legg inn kode her for å vise venn fra Skjermbilde 1 i listen
       som vises i Skjermbilde 2 */
  }
}

const felt1 = document.getElementById("registrering");
const felt2 = document.getElementById("visning");

new Vennkontroller(felt1, felt2);
```

Oppgave 6 (40%) - Web back-end med Spring MVC

Se oppgave på papir. For å svare, velger du «Administrer vedlegg». Så

Nytt vedlegg
Kode
Kodespråk - Java

Da får du åpnet en ny fane der du skriver koden.

For å sikre automatisk lagring, følg denne oppskriften

1. Start med å gi vedlegget et navn. For eksempel "Oppgave 6"
2. Klikk lagre. Da aktiverer du automatisk lagring.
3. Start å skrive kode

Når du har svart, går du tilbake til hovedfanen.

</>